

Web Application Attack Lab Report

Josh Smith | April 2026

1. Lab Overview

This report documents the setup, execution, and detection of web application attacks conducted in a home SOC lab environment. The target was DVWA (Damn Vulnerable Web Application), running on an Ubuntu 26.04 VM with an Apache web server. Attacks were carried out from a Kali Linux VM on the same bridged network. All traffic was logged by Apache and shipped to Elasticsearch via Filebeat for analysis and detection in Kibana.

2. Lab Environment

2.1 Hardware

Apple Mac Mini M4, 32GB RAM, macOS. All virtual machines run under UTM on a bridged network, placing each VM on the same subnet as the host.

2.2 Virtual Machines

VM	IP Address	OS	Role
Ubuntu-SIEM	192.168.1.58	Ubuntu 26.04 ARM64	Target / SIEM host
Kali Linux	192.168.1.18	Kali 2023 ARM64	Attack platform
Mac Mini (host)	192.168.1.68	macOS	Browser-based attacks

2.3 Stack

- Elasticsearch 8.19.14
- Kibana 8.19.14
- Filebeat 8.19.14 with Apache module
- Apache 2.4.66 logging to /var/log/apache2/access.log
- DVWA (Damn Vulnerable Web Application) installed at <http://192.168.1.58/DVWA/>
- MySQL backend for DVWA

3. Lab Setup

3.1 DVWA Configuration

DVWA was installed on the Ubuntu-SIEM VM alongside Apache and MySQL. The DVWA security level was set to Low to allow all attack types without built-in mitigations. The MySQL DVWA user was configured with the password Password123 and the database was initialized via the DVWA Setup page.

3.2 Log Collection Pipeline

Getting Apache logs into Kibana required resolving a Filebeat configuration issue. Filebeat was installed alongside the ELK stack but was attempting to connect to Elasticsearch over plain HTTP while Elasticsearch 8.x defaults to HTTPS. This caused all events to queue up in memory without being shipped.

The fix involved two changes to `/etc/filebeat/filebeat.yml`:

- Changing the output host from `http://localhost:9200` to `https://localhost:9200`
- Adding Elasticsearch credentials and disabling SSL verification for the local self-signed certificate

After restarting Filebeat, 3,200 queued events flushed to Elasticsearch within seconds. The Apache module was then enabled and Filebeat was configured to harvest `/var/log/apache2/access.log`. All subsequent attack traffic was ingested in near real-time under the `filebeat-*` index pattern.

4. Attacks Performed

Six attack types were performed against DVWA across two attack surfaces: the browser from the Mac host and automated tools from Kali Linux.

4.1 SQL Injection (Manual)

Performed via the DVWA SQL Injection module using the browser. Two payloads were submitted through the user ID field:

- `1' OR '1'='1` — bypassed the WHERE clause and returned all user records
- `1' UNION SELECT user, password FROM users#` — extracted MD5 hashes for all accounts

The admin hash `5f4dcc3b5aa765d61d8327deb882cf99` was identified as the MD5 value of the plaintext password, confirming the injection succeeded and sensitive data was exposed.

4.2 Command Injection (Manual)

Performed via the DVWA Command Injection module. The ping field was manipulated to chain additional OS commands using semicolons:

- 192.168.1.58; whoami — confirmed execution as www-data
- 192.168.1.58; cat /etc/passwd — dumped the full system user list

The server executed both commands and returned the output in the HTTP response, confirming unsanitized input was being passed directly to a shell.

4.3 XSS Reflected (Manual)

Performed via the DVWA XSS (Reflected) module. The following payload was submitted in the name input field:

- `<script>alert('XSS')</script>`

The browser executed the script and displayed an alert dialog, confirming the application reflected the input back to the page without sanitization. Stored XSS was not tested as the module returned HTTP 500 due to a PHP configuration issue unrelated to this lab.

4.4 Web Scanner (Nikto)

Nikto was run from Kali against the DVWA base directory. The scan completed in 6 seconds and generated 8,102 HTTP requests, producing 16 findings including:

- Exposed /DVWA/config/ directory with directory indexing enabled
- Exposed /DVWA/database/ directory
- Admin login page identified at /DVWA/login.php
- Missing X-Frame-Options and X-Content-Type-Options security headers
- Exposed .git directory and associated files (index, HEAD, config, .gitignore)
- Nikto also probed for CVE-2014-6271 (Shellshock), which appeared in Apache logs via the user agent string

The Nikto scan was the single largest source of traffic in the lab, accounting for the majority of the 9,401 total requests logged from 192.168.1.18.

4.5 Brute Force (Hydra)

Hydra was run from Kali against the DVWA login page using the rockyou.txt wordlist. The attack used the http-post-form module targeting the username and password fields. Hydra found valid credentials on the 8th attempt:

- Username: admin
- Password: rockyou

The attack completed in under 15 seconds, generating 239 POST requests to /DVWA/login.php visible in Apache logs. The rapid volume of POST requests from a single IP is a reliable brute force indicator.

4.6 Automated SQL Injection (SQLmap)

SQLmap was run from Kali with a valid DVWA session cookie and level 2 testing. It identified three injection techniques against the id GET parameter:

- Boolean-based blind injection
- Time-based blind injection (SLEEP)
- UNION query injection (2 columns)

SQLmap confirmed the backend DBMS as MySQL and enumerated all three databases: dvwa, information_schema, and performance_schema.

5. Attack Summary

Attack	Tool	Source	Result
SQL Injection	Browser	Mac host	Dumped all users and MD5 hashes
Command Injection	Browser	Mac host	RCE as www-data, dumped /etc/passwd
XSS Reflected	Browser	Mac host	Script executed, alert displayed
Web Scanner	Nikto	Kali 192.168.1.18	16 findings, 8,102 requests
Brute Force	Hydra	Kali 192.168.1.18	Credentials found in 8 attempts
SQLmap	SQLmap	Kali 192.168.1.18	3 injection types, 3 DBs enumerated

6. Detection Rules

Three detection rules were created in Kibana under Security > Detection Rules, all running on a 1-minute schedule with a 5-minute look-back window against the filebeat-* index.

6.1 Web Scanner Detected

Rule type: Threshold. Triggers when a single source IP generates more than 100 requests against the DVWA application within 1 minute. This catches automated scanner behavior like Nikto which generates thousands of requests in seconds. In production this rule would be paired with an automatic firewall block via UFW or a WAF rule.

6.2 SQL Injection Attempt

Rule type: Custom query. Triggers on any request where the URL contains SQL keywords including SELECT, UNION, OR+1=1, or single quote characters. The rule caught 166 events from the manual and automated injection tests. Alert triage should prioritize any events with HTTP 200 response codes as these indicate potentially successful injection.

6.3 Brute Force Login

Rule type: Threshold. Triggers when a single source IP sends more than 10 POST requests to /login.php within 1 minute. Hydra generated 239 POST requests in under 15 seconds, well above the threshold. Recommended response is immediate IP block and audit of the targeted account for any successful authentication following the burst.

7. Kibana Dashboard

The Web App Attack Dashboard was built in Kibana using the filebeat-* index filtered to event.module: apache. It contains six panels:

Panel	What It Shows
Web Attack Traffic Over Time	Bar chart split by source IP showing the Nikto spike at 18:00
Top Attacking IPs	Kali at 9,401 requests vs Mac host at 151
HTTP Response Codes	200s, 302s (login redirects), 404s (Nikto probing), 500s
HTTP Methods	Pie chart showing GET at 98.83% and POST at 0.97%
Top Requested URLs	Login page at 239 hits, DVWA root, setup page
Top User Agents	sqlmap/1.7.8, Nikto/2.5.0, CVE-2014-6271 probe string

8. Key Findings

The following findings came out of the attack and detection analysis:

- The DVWA application had no input validation on any tested parameter, making SQL injection and command injection trivial to exploit from a browser with no tools required.
- The admin account used a weak, commonly known password that appeared in the rockyou wordlist within the first 10 attempts. No account lockout policy was in place.
- Nikto identified an exposed .git directory including the HEAD file and config, which in a real environment would allow an attacker to reconstruct the entire source repository.
- The config and database directories were accessible without authentication and had directory indexing enabled, exposing database connection strings and schema files.
- SQLmap confirmed three distinct injection techniques against the same parameter, indicating the application provided no filtering at any layer.

- Shellshock (CVE-2014-6271) was probed by Nikto and appeared in Apache logs via the user agent string, demonstrating how scanner artifacts persist in logs for forensic analysis.
- The Filebeat pipeline required a non-trivial fix to function. The default configuration shipped HTTP credentials to an HTTPS-only Elasticsearch instance, causing silent log loss for over three hours. This highlights the importance of validating log pipeline health as part of any SIEM deployment.

9. Recommendations

The following controls would eliminate or significantly reduce the risk of the attack vectors demonstrated in this lab:

- Input validation and parameterized queries should be implemented for all database interactions to prevent SQL injection. Prepared statements remove the risk entirely.
- OS command execution should never use unsanitized user input. Where shell commands are necessary, allowlists of permitted values should be enforced.
- Output encoding should be applied to all user-supplied data rendered in HTML to prevent XSS.
- Account lockout policies should trigger after 5 to 10 failed login attempts, blocking brute force tools like Hydra from completing a full wordlist attack.
- Exposed directories including .git, config, and database should be blocked via Apache configuration. Directory indexing should be disabled globally.
- Security headers including X-Frame-Options, X-Content-Type-Options, and Content-Security-Policy should be added to all responses.
- In a production environment, the three Kibana detection rules created in this lab would be connected to a SOAR platform or ticketing system such as ServiceNow for automated alerting and response workflow.

10. Conclusion

This lab demonstrated the full attack lifecycle against a vulnerable web application, from reconnaissance with Nikto through data exfiltration with SQLmap, and built a functional detection layer on top of the resulting log data. The lab produced six categories of confirmed attack traffic, three Kibana detection rules, and a six-panel SOC dashboard.

The most significant technical challenge was resolving the Filebeat pipeline failure caused by a protocol mismatch between the default HTTP output configuration and Elasticsearch's HTTPS requirement. This introduced a real-world troubleshooting scenario and reinforced the importance of validating end-to-end log ingestion before running any lab exercises.

Lab artifacts including Kibana exports, detection rule configurations, and dashboard screenshots are published at github.com/jsmith0622/web-app-attack-lab.